

Modeling Industrial ADMET Data with Multitask Networks

Steven Kearnes
Stanford University
kearnes@stanford.edu

Brian Goldman
Vertex Pharmaceuticals Inc.
brian_goldman@vrtx.com

Vijay Pande
Stanford University
pande@stanford.edu

Abstract

Deep learning methods such as multitask neural networks have recently been applied to ligand-based virtual screening and other drug discovery applications. Using a set of industrial ADMET datasets, we compare neural networks to standard baseline models and analyze multitask learning effects with both random cross-validation and a more relevant temporal validation scheme. We confirm that multitask learning can provide modest benefits over single-task models and show that smaller datasets tend to benefit more than larger datasets from multitask learning. Additionally, we find that adding massive amounts of side information is not guaranteed to improve performance relative to simpler multitask learning. Our results emphasize that multitask effects are highly dataset-dependent, suggesting the use of dataset-specific models to maximize overall performance.

1 Introduction

The 2012 Merck Molecular Activity Challenge [Dahl, November 1, 2012] catalyzed a surge of interest in using artificial neural networks and “deep learning” [LeCun et al., 2015] for problems in drug discovery and cheminformatics, especially virtual screening. Follow-up studies have shown that neural networks, on average, outperform traditional machine learning methods such as random forest [Dahl et al., 2014; Ma et al., 2015; Ramsundar et al., 2015; Mayr et al., 2015].

The winners of the Merck challenge utilized multitask neural networks (MTNNs), which are trained to predict many outputs simultaneously in order to improve performance relative to single-task models [Caruana, 1997]. As shown in Figure 1, MTNNs

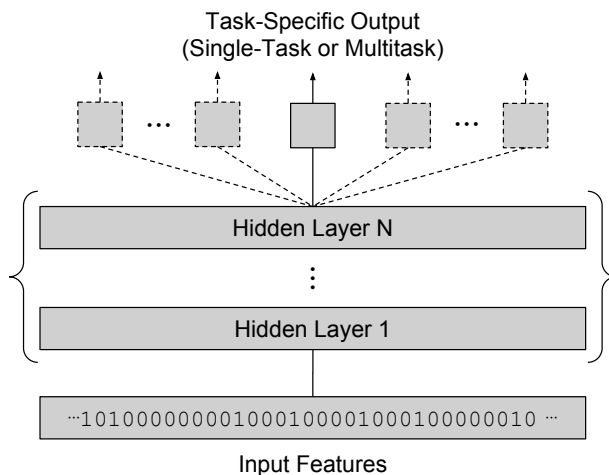


FIGURE 1: Abstract neural network architecture. The input vector is a binary molecular fingerprint with 1024 bits. All connections between layers are *dense*, meaning that every unit in layer n is connected to every unit in layer $n + 1$. Each output block is a task-specific two-class softmax layer; dashed lines indicate that models can be either single-task or multitask.

share a subset of model parameters between all tasks, encouraging the model to learn an internal representation of the input data that is useful for all tasks. If the tasks in a multitask model are related to one another, the parameters in the hidden representation are effectively exposed to more training data than they would be in a single-task model with the same architecture. This “data amplification” effect—possibly in combination with additional effects described by Caruana [1997]—helps to explain how multitask models can outperform their single-task analogs (the so-called *multitask effect*).

Although multitask learning improves performance on average, it is not clear what factors contribute to multitask improvement for specific datasets. One

major issue is that task *relatedness* is not well defined. For example, Ma et al. [2015] observed that models trained on the largest datasets in their collection, which had many molecules in common, performed *worse* in a multitask setting. Additionally, the amount of data required for multitask learning is not well understood, nor is it possible to predict when training with additional data or tasks will significantly improve performance. Some studies using public data have emphasized the addition of large amounts of side information to improve multitask performance [Ramsundar et al., 2015; Unterthiner et al., 2014], leading to the impression that more data is better (on average) with little dependence on apparent task relatedness or dataset size.

Apart from uncertainty surrounding the multitask effect, it is possible that results based on publicly available data will not transfer well to industrial settings. Besides data quality, the most obvious potential issue is the use of random cross-validation instead of temporal validation. Sheridan [2013] demonstrated that random cross-validation generally gives overly optimistic estimates of prospective performance, but temporal validation is often impossible when working with public data due to the lack of temporal metadata (timestamps). In this context, Ma et al. [2015] used temporally annotated industrial data to demonstrate that neural networks can outperform random forest models (they also performed some experiments with multitask networks).

This report describes applications of single-task and multitask neural networks to absorption, distribution, metabolism, excretion, and/or toxicity (ADMET) assay data in an industrial drug discovery setting at Vertex Pharmaceuticals. Using a selection of our internal datasets, our main objectives are to (1) compare single-task and multitask neural network models to baseline models including random forest and logistic regression, (2) assess the impact of temporal validation *vs.* random cross-validation on model performance, and (3) explore several factors that could potentially influence multitask effects. Additionally, we consider the problem of information leakage in multitask models. Our results confirm previous work showing that multitask learning can improve model performance while emphasizing the fact that multitask effects are highly dataset-dependent.

2 Methods

2.1 Datasets

Our experiments focused on classification performance for a set of 22 Vertex datasets with various ADMET endpoints including hERG inhibition, aqueous solubility, compound metabolism, and others. Experimental data points were divided into “active”, “inconclusive”, and “inactive” classes using dataset-specific thresholds, and “inconclusive” results were discarded. These datasets contained approximately 280 000 experimental values and are described in Table 1. Additional data used as side information (see Section 3.2.2) are described in Section A.1.

Since many of the tasks have imbalanced active/inactive proportions, we weighted the examples in the minority class such that the active and inactive classes had equal weight. Active/inactive ratios were calculated using training data (except for random cross-validation, where the ratio was determined with the full dataset). Note that these example weights were not used in logistic regression or random forest models.

Models were trained using a temporal validation scheme: each data point has an associated experiment date, and divisions into training, validation, and test set divisions were based on temporally ordered date ranges as in Sheridan [2013]. Date cut-offs were chosen to divide the data for each dataset into approximately 70% training, 10% validation, and 20% test. For comparison to temporal models, we also trained models using a random cross-validation scheme where each dataset was randomly partitioned into five folds in a stratified manner such that the active/inactive ratio of the full dataset was approximately preserved in each fold.

Dataset compounds were represented by 1024-bit circular fingerprints with radius 2 (similar to the ECFP4 fingerprints described by Rogers and Hahn [2010]) generated with the OpenEye GraphSim Toolkit.

2.2 Model training and evaluation

Neural network models were constructed with TensorFlow [Abadi et al., 2015], an open source library for machine learning. We used standard feed-forward architectures with rectified linear activations, batch normalization [Ioffe and Szegedy, 2015], 0.5 dropout [Srivastava et al., 2014], and the adagrad optimizer [Duchi et al., 2011] with learning rate 0.001

TABLE 1: Proprietary datasets used for model evaluation. Each data point is associated with an experiment date used for temporal validation.

Dataset	Actives	Inactives	Total
A	20 247	9652	29 899
B	32 806	23 936	56 742
C	40 136	27 703	67 839
D	24 379	2374	26 753
E	21 722	2746	24 468
F	25 202	2034	27 236
G	2003	3226	5229
H	500	526	1026
I	669	344	1013
J	883	399	1282
K	845	357	1202
L	489	164	653
M	820	357	1177
N	1420	740	2160
O	670	1417	2087
P	3861	4107	7968
Q	1056	2658	3714
R	215	2760	2975
S	987	582	1569
T	1454	5935	7389
U	3998	2790	6788
V	2795	896	3691
	187 157	95 703	282 860

and batch size 128. Models were trained for 1 M steps (except as otherwise noted) with periodic checkpointing. Input examples were encoded in a “dense” format, such that there was one training example for each unique molecule identifier containing features for that molecule and labels and weights for each task. For tasks where an example did not have activity data, the weight was set to zero. Note that we did not use SMILES strings as molecule identifiers, so some molecules with identical SMILES may have been represented by multiple training examples. Models were trained on a cluster with several NVIDIA Tesla K80 GPUs, although no multi-GPU training was performed.

Each neural network model was based on one of five chosen architectures. Each architecture is specified using $(x_1, x_2, \dots, x_n)$ notation, where the input is fed into a fully-connected layer with x_1 units, followed by a layer with x_2 units, and so on. The (1000) architecture represents a reasonable baseline network. The (2000, 100) and (2000, 1000) architectures examine how the size of the final representation affects learning; Ramsundar et al. [2015] hypothe-

sized that a pyramidal architecture with relatively few task-specific parameters helps to avoid overfitting while still providing a rich shared representation. The (4000, 2000, 1000, 1000) architecture was recommended by Ma et al. [2015] for regression on chemical datasets, and the (4000) architecture investigates the use of a single wide layer in lieu of multiple hidden layers.

Logistic regression and random forest baselines were built using the `LogisticRegression` and `RandomForestClassifier` classes, respectively, in scikit-learn [Pedregosa et al., 2011]. Parameters for random forest models were similar to those described by Ma et al. [2015]; specifically `n_estimators=100`, `max_features=1/3.`, and `min_samples_split=6`. We note that these parameters were selected for different molecular descriptors and may not be optimal for use with circular fingerprints.

Model performance was evaluated using the area under the receiver operating characteristic curve (ROC AUC, or simply AUC), which is a global measure of classification performance [Jain and Nicholls, 2008]. For evaluation of neural network models, a single training checkpoint was selected that maximized the validation set AUC for the task of interest, and this checkpoint was used to make predictions and calculate AUC scores for test set compounds. In practice, this meant that multitask models were often evaluated at several different training checkpoints that optimized validation set performance for individual tasks. We note that some tasks achieved their best validation score near the beginning or end of training and therefore some reported results may be subject to overfitting or underfitting, respectively.

Although ROC AUC is the recommended metric for evaluating virtual screening models [Jain and Nicholls, 2008], it is a global metric that is not always useful for making decisions in a production setting. AUC values reflect the relative positions of active and inactive compounds in a ranked list of predicted values, but compound-specific decisions require the choice of a threshold on predicted values and estimates of prediction confidence. Additionally, “enrichment” scores based on ROC curves can be used to measure performance early in a ranked list [Jain and Nicholls, 2008]. The analysis that follows is based on ROC AUC scores; explorations of alternative metrics or strategies for choosing decision thresholds, estimating prediction confidence, and/or determining the domain of applicability for models are beyond the scope of this report.

2.3 Model comparison

For comparisons between models we report the median ΔAUC across the datasets in Table 1 and a 95% confidence interval for the sign test statistic. The sign test is a paired non-parametric test that measures the fraction of per-dataset ΔAUC values that are greater than zero (exactly zero differences are excluded). The 95% confidence interval is a Wilson score interval around this fraction that estimates the probability that one model will outperform another (in terms of AUC). Confidence intervals that do not include 0.5 indicate statistically significant differences; i.e. two models are statistically indistinguishable if there is not a clear bias toward one model or the other. Conceptually, the sign test confidence interval estimates the consistency of observed differences between models, while the median ΔAUC estimates the effect size. Confidence intervals were calculated with the `proportion_confint` method in statsmodels [Seabold and Perktold, 2010] using `alpha=0.05` and `method='wilson'`.

3 Results

3.1 Model performance

We trained single-task neural network (STNN) and multitask neural network (MTNN) models on our dataset collection using a temporal validation scheme. We used two flavors of MTNN: standard MTNNs used uniform weights for the cost associated with each task (U-MTNN), while task-weighted MTNNs assigned weights to each task that were inversely proportional to the number of training compounds for each task, such that the total weight for each task was approximately equal (W-MTNN). This approach allowed us to investigate whether it is important to upweight small tasks in order for them to benefit from multitask learning. Additionally, NN models (but not logistic regression or random forest models) used per-example weights that attempted to compensate for imbalance between actives and inactives in each dataset (see Section 2.1).

Median test set AUC values for these models are reported in Table 2, along with values for random forest and logistic regression baselines. For comparisons between models, we report median ΔAUC values and sign test 95% confidence intervals (see Section 2.3). MTNN models had consistently better performance than random forest regardless of architecture (note that “consistent” does not necessarily



FIGURE 2: Box plots showing ΔAUC values between MTNN and STNN models with the same core architecture. Each box plot summarizes 10 ΔAUC values, one for each combination of model architecture (e.g. (2000, 1000)) and task weighting strategy (U-MTNN or W-MTNN).

imply superior performance for all datasets). Surprisingly, neural networks were not as robust when compared to logistic regression; the (2000, 100) and (4000, 2000, 1000, 1000) MTNNs were statistically indistinguishable from the logistic regression baseline. However, the (1000), (4000), and (2000, 1000) MTNN models outperformed logistic regression, with task-weighted models giving the most consistent improvements as measured by sign test confidence intervals.

Table 3 compares single-task and multitask neural network models with the same core (hidden layer) architecture. Of the MTNN models we trained, only the (1000) and (2000, 1000) W-MTNN models showed significant improvement over their single-task counterparts. There were no consistent differences between U-MTNN and W-MTNN models (we investigate dataset-specific effects of per-task weighting in Section 3.2.1).

Although multitask effects varied between models, the effects observed for individual datasets were relatively independent of model architecture or task weighting strategy (Figure 2).

Table A2 shows pairwise comparisons between model architectures in the same class (e.g. STNN). No architecture gave significant improvements over all others, suggesting that the effectiveness of any

TABLE 2: Median test set AUC values for random forest, logistic regression, single-task neural network (STNN), and multitask neural network (MTNN) models. U-MTNN models treat each task uniformly; W-MTNN models are task-weighted models, meaning that the cost for each task is weighted inversely proportional to the amount of training data for that task. We also report median Δ AUC values and sign test 95% confidence intervals for comparisons between each model and random forest or logistic regression (see Section 2.3). Bold values indicate confidence intervals that do not include 0.5.

Model	Median AUC	Model - Random Forest		Model - Logistic Regression	
		Median Δ AUC	Sign Test 95% CI	Median Δ AUC	Sign Test 95% CI
Random Forest	0.719			−0.016	(0.20, 0.57)
Logistic Regression	0.758	0.016	(0.43, 0.80)		
STNN	(1000)	0.748	(0.47, 0.84)	0.007	(0.39, 0.77)
	(4000)	0.761	(0.52, 0.87)	0.015	(0.52, 0.87)
	(2000, 100)	0.749	(0.47, 0.84)	0.007	(0.35, 0.73)
	(2000, 1000)	0.759	(0.47, 0.84)	0.008	(0.35, 0.73)
	(4000, 2000, 1000, 1000)	0.736	(0.43, 0.80)	−0.011	(0.27, 0.65)
U-MTNN	(1000)	0.792	(0.67, 0.95)	0.029	(0.52, 0.87)
	(4000)	0.768	(0.61, 0.93)	0.031	(0.57, 0.90)
	(2000, 100)	0.797	(0.61, 0.93)	0.023	(0.43, 0.80)
	(2000, 1000)	0.800	(0.67, 0.95)	0.040	(0.52, 0.87)
	(4000, 2000, 1000, 1000)	0.809	(0.72, 0.97)	0.024	(0.43, 0.80)
W-MTNN	(1000)	0.793	(0.78, 0.99)	0.040	(0.67, 0.95)
	(4000)	0.773	(0.72, 0.97)	0.036	(0.67, 0.95)
	(2000, 100)	0.769	(0.61, 0.93)	0.022	(0.43, 0.80)
	(2000, 1000)	0.821	(0.78, 0.99)	0.041	(0.67, 0.95)
	(4000, 2000, 1000, 1000)	0.800	(0.61, 0.93)	0.035	(0.47, 0.84)

TABLE 3: Comparisons between neural network models. Differences between STNN, U-MTNN, and W-MTNN models with the same core (hidden layer) architecture are reported as median Δ AUC values and sign test 95% confidence intervals. Bold values indicate confidence intervals that do not include 0.5.

Model		MTNN - STNN		W-MTNN - U-MTNN	
		Median Δ AUC	Sign Test 95% CI	Median Δ AUC	Sign Test 95% CI
U-MTNN	(1000)	0.010	(0.43, 0.80)		
	(4000)	0.012	(0.43, 0.80)		
	(2000, 100)	0.015	(0.39, 0.77)		
	(2000, 1000)	0.026	(0.47, 0.84)		
	(4000, 2000, 1000, 1000)	0.023	(0.43, 0.80)		
W-MTNN	(1000)	0.017	(0.52, 0.87)	0.002	(0.37, 0.76)
	(4000)	0.007	(0.47, 0.84)	0.002	(0.35, 0.73)
	(2000, 100)	0.004	(0.39, 0.77)	−0.002	(0.28, 0.68)
	(2000, 1000)	0.032	(0.57, 0.90)	0.005	(0.43, 0.80)
	(4000, 2000, 1000, 1000)	0.033	(0.43, 0.80)	0.004	(0.43, 0.80)

given architecture is highly dependent on the available data. As such, we cannot draw any general conclusions about the utility of one architecture over another except to say that the (2000, 1000) W-MTNN architecture achieved the highest median AUC for our datasets and consistently improved upon logistic regression, random forest, and (2000, 1000) STNN models.

3.2 Factors affecting multitask learning

Several reports have shown that, on average, multitask learning improves performance relative to single-task models [Dahl et al., 2014; Ma et al., 2015; Ramsundar et al., 2015; Mayr et al., 2015]. However, these studies also include examples of datasets that see only small or even negative effects in a multitask setting. In this section, we investigate several factors that may contribute directly or indirectly to observed multitask effects.

3.2.1 Individual dataset size

It is possible that the size of individual datasets affects multitask performance. Figure 3 shows a plot of multitask benefit *vs.* dataset size for the (2000, 1000) W-MTNN from Table 2. There is a slight negative trend, suggesting that larger datasets benefit less from multitask training, possibly because they have enough data to generate a useful hidden layer representation without additional side information. These results confirm previous work showing multitask improvement on relatively small datasets [Dahl et al., 2014; Ma et al., 2015; Ramsundar et al., 2015] and should be encouraging since pharmaceutical datasets are usually much smaller than those used for other deep learning applications.

As an additional experiment with dataset size, we trained multitask models for the “small” datasets in our collection with fewer than 10 000 data points (datasets G–V) to test whether small datasets were overwhelmed by larger tasks despite the use of per-task weighting. Table A3 shows comparisons between models trained using only small datasets and full (22 task) multitask models for the 16 small datasets. We did not observe consistent improvements when training only on small datasets, although some models gave improvements for individual datasets (Figure A1).

We also considered the possibility that U-MTNN and W-MTNN models did not have significantly



FIGURE 3: Multitask benefit as a function of dataset size (log scale, training set only; $r^2 \approx 0.12$). Multitask benefit for each task is calculated as the difference in AUC between the (2000, 1000) W-MTNN and (2000, 1000) STNN models.

different performance due to size-dependent effects; e.g. task weighting may improve performance on small datasets but hurt performance on large datasets, producing a net effect that appears insignificant. However, we did not observe a size-dependent trend when we compared U-MTNN and W-MTNN performance on individual datasets (Figure A2).

3.2.2 Total amount of data

Deep learning models have many parameters and perform best when they are trained on huge amounts of data (typically millions of examples). The simplest neural network models in our experiments have a single hidden layer with 1000 units, requiring over 1 M learned parameters (not including the task-specific softmax heads). In contrast, a multitask model trained on all of the datasets in our collection has fewer than 200 000 training set data points, which may simply not be enough data to build effective models or control overfitting (despite the use of dropout).

In the spirit of previous work that utilized large amounts of public data to enhance multitask learning [Ramsundar et al., 2015; Unterthiner et al., 2014], we constructed a side information dataset using internal Vertex data as well as public sources (see Section A.1 for details). In combination with our original datasets, this larger collection contained approximately 38 M data points divided across more than 550 tasks. If multitask effects can be modulated by

the total amount of data and/or tasks, we reasoned that this larger dataset collection should significantly improve the average performance on the datasets in Table 1. Training each model on this larger dataset collection required several weeks with a single GPU (all models were trained for at least 40 M steps). We note that some validation set AUC scores were still increasing when training was stopped, so there is some risk of underfitting in the results that follow; training curves for the tasks in our dataset collection are shown in Figure A3 and Figure A4.

Table A4 reports performance on the datasets from Table 1 for models trained with additional side information. Notably, the (2000, 1000) and (4000, 2000, 1000, 1000) W-MTNN models trained with side information achieved some of the highest median AUC scores of all reported models and consistently improved upon STNN models with the same core architecture (Table A5). Additionally, per-task weighting led to significant performance improvements for most architectures. However, adding side information did not consistently improve performance relative to the MTNN models reported above; comparisons between multitask models trained with and without additional side information showed that, on average, adding side information either had no significant effect or damaged performance (Table 4).

Inspection of per-dataset differences revealed that the effect of adding side information was dataset-dependent (Figure A5). In contrast to the trend observed in Figure 3, some of the largest datasets in our collection benefited the most from additional side information, including three datasets (D, E, and F) with related targets. These datasets had additional related assays in the included side information, suggesting that these improvements might be due to additional related training examples rather than an increase in the total amount of data.

3.2.3 Task relatedness

It is possible that the datasets in our collection are diverse enough that naïvely combining them into a single multitask model introduces competing training signals that dampen multitask effects (see Caruana [1997] for a more thorough analysis of multitask learning). To test this hypothesis, we trained multitask models using subsets of the datasets in our collection that are related by similar targets and are therefore most likely to benefit from multitask learning (Table A6). We note that Erhan et al. [2006] also constructed models using subsets of related tasks, us-

ing pairwise correlations between labels for shared compounds as a measure of task similarity.

Table A7 shows comparisons between subset and full (22 task) multitask models for the 10 datasets used to train subset models. These comparisons skew toward worse performance for subset models, but none of the differences are statistically significant. Importantly, the subset models had fewer training examples than full multitask models, and it is possible that any additional multitask benefits due to more deliberate task selection were dampened by training with less data. Box plots summarizing per-dataset differences between subset and full models are given in Figure A6.

These results support the inclusion of as much data and as many tasks as possible when training multitask models, without regard to task relatedness. In light of the seemingly contradictory result from Section 3.2.2, we suggest that multitask benefits increase with the addition of more data up to an inflection point where additional data does not help or even begins to degrade performance. This inflection point is likely to vary between datasets and will depend on relatedness to the other tasks in the model.

3.3 Information leakage in multitask networks

Multitask models have the potential for *information leakage* across tasks, which can lead to overly optimistic validation results even when using temporal validation. Information leakage occurs when the training set for one task is unrealistically related to the test set for another task. We use the word “unrealistically” to suggest that there are situations where relatedness is not unfair, as might occur when starting a new screening campaign against a target that is very similar to one that has been screened previously. (We note that information leakage in multitask networks was treated briefly by Ramsundar et al. [2015] when considering the effect of random cross-validation on closely related tasks.)

Consider a multitask model trained using a temporal validation scheme. If each dataset is divided into training and test data independently, it is possible that training data for one task was generated in the future relative to the training data for a second task. This would allow the second task to benefit from information that should not realistically be available to that task and partially defeats the purpose of temporal validation. We refer to this as “leaky” valida-

TABLE 4: Comparisons between multitask models trained with and without additional side information (SI). We report the median ΔAUC and sign test 95% confidence interval comparing MTNN+SI and MTNN models with the same core architecture and task weighting strategy. Bold values indicate confidence intervals that do not include 0.5.

Model		MTNN+SI - MTNN	
		Median ΔAUC	Sign Test 95% CI
U-MTNN	(1000)	-0.035	(0.07, 0.39)
	(4000)	0.004	(0.35, 0.73)
	(2000, 100)	-0.041	(0.10, 0.43)
	(2000, 1000)	-0.020	(0.16, 0.53)
	(4000, 2000, 1000, 1000)	-0.027	(0.20, 0.57)
W-MTNN	(1000)	-0.009	(0.23, 0.61)
	(4000)	0.000	(0.31, 0.69)
	(2000, 100)	0.001	(0.31, 0.69)
	(2000, 1000)	0.002	(0.31, 0.69)
	(4000, 2000, 1000, 1000)	0.007	(0.39, 0.77)

tion. Random cross-validation is always leaky (and maximally so), because there is no respect to past or future whatsoever. The impact of information leakage is expected to be proportional to task relatedness, such that more-related tasks should benefit more from leaked information than less-related tasks.

Information leakage can be prevented by constructing models with a definition of past and future that is consistent across all tasks, i.e. selecting a single point in time that separates training and test data for all tasks. We refer to this strategy as “non-leaky” temporal validation. Figure 4 presents graphical descriptions of leaky and non-leaky temporal validation.

We constructed non-leaky multitask models on our datasets for comparison with the leaky multitask models presented above. Due to different date ranges and dataset sizes, a single non-leaky model would give very poor statistics for many tasks due to very small training or test sets. To correct for this issue, and to make leaky and non-leaky models more directly comparable, we constructed *dataset-specific* non-leaky models. For each dataset in turn (the “focus dataset”), the training, validation, and test set cutoff dates used for single-task and leaky multitask models were applied to the remaining datasets to construct non-leaky multitask training, validation, and test sets, respectively. As a result, the focus dataset in each non-leaky model had the same training, validation, and test sets as its single-task and leaky counterparts without including anachronistic side information. We trained each non-leaky multitask model for 1 M steps and measured the test set performance

for the focus dataset using the training checkpoint that maximized performance on the validation set for the focus task (data for the other tasks were used only as side information during training).

Table A8 reports the performance of the non-leaky multitask models relative to random forest and logistic regression baselines. Median AUC values for several models were quite different from the leaky results in Table 2, but the general trends in performance were consistent for both validation strategies. These results suggest that some information leakage occurred with leaky validation on our dataset collection, leading to higher median AUC values for some models (especially the (2000, 1000) and (4000, 2000, 1000, 1000) architectures). Comparisons between neural network models showed that some non-leaky models outperformed their leaky counterparts relative to single-task models (compare Table 3 and Table A9), and per-task weighting significantly improved performance for the (4000, 2000, 1000, 1000) architecture. These results seem counterintuitive, but would not be unexpected in a situation where the non-leaky cutoff dates assigned *all* of the data for one or more side information tasks to the training set. However, it is unexpected that the changes in median AUC were not consistently in the same direction, given that all non-leaky models were trained with the same data. Direct comparisons between non-leaky and leaky models did not reveal any significant differences in performance between the two validation strategies (Table 5).

We caution that these results are specific to our dataset collection and may not hold for datasets with

TABLE 5: Comparisons between leaky and non-leaky multitask models. We report the median ΔAUC and sign test 95% confidence interval comparing non-leaky and leaky models with the same core architecture and task weighting strategy. Bold values indicate confidence intervals that do not include 0.5.

Model		Non-leaky - Leaky	
		Median ΔAUC	Sign Test 95% CI
U-MTNN	(1000)	0.001	(0.35, 0.73)
	(4000)	0.001	(0.35, 0.73)
	(2000, 100)	-0.002	(0.23, 0.61)
	(2000, 1000)	-0.010	(0.20, 0.57)
	(4000, 2000, 1000, 1000)	-0.005	(0.20, 0.57)
W-MTNN	(1000)	0.000	(0.31, 0.69)
	(4000)	-0.003	(0.23, 0.61)
	(2000, 100)	0.002	(0.31, 0.69)
	(2000, 1000)	-0.011	(0.20, 0.57)
	(4000, 2000, 1000, 1000)	0.002	(0.35, 0.73)

different date ranges, relatedness, or amounts of data. In particular, the distribution of experiment dates for a dataset collection can lead to very different amounts of training data for non-leaky and leaky models. Our use of leaky validation for the majority of the analyses in this report avoids sensitivity to the specific date ranges for our datasets—simplifying the exploration of multitask effects—at the risk of unrealistic information leakage.

3.4 Temporal validation *vs.* random cross-validation

Sheridan [2013] has shown that temporal validation gives more accurate estimates of prospective model performance than random cross-validation in drug discovery applications. Random cross-validation reduces *covariate shift*—the tendency for training and test examples to follow different distributions. Covariate shift is especially common in pharmaceutical data, since compounds are synthesized serially as project teams attempt to optimize potency, off-target effects, and other molecular properties [McGaughey et al., 2016]. In the worst case, compounds used for training and test data in temporal validation could have entirely different properties. By ignoring the temporal evolution of a dataset, random cross-validation makes it much more likely that the training data is a good approximation of the test data—a standard assumption in machine learning—but a poor approximation of reality.

As a measure of covariate shift in our dataset col-

lection, we measured the maximum circular fingerprint Tanimoto similarity between each test compound and all training set compounds in each dataset (note that temporal validation set compounds were excluded). Histograms of maximum similarities for temporal validation and random cross-validation are shown in Figure 5. As expected, test compounds were much more likely to have a highly similar training compound when using random cross-validation.

To compare the performance of models trained using temporal validation or random cross-validation, we trained models using 5-fold random cross-validation and measured 5-fold mean test set AUC values after training for ~ 1 M steps. Table A11 shows that random cross-validation multitask models consistently improved upon logistic regression and random forest baselines. Interestingly, the logistic regression and random forest models also switched their relative performance compared to temporal validation. Table A12 shows that random cross-validation multitask networks consistently beat their single-task counterparts regardless of architecture or task weighting strategy. Overall, multitask effects for random cross-validation models were more consistent than for temporal models, although the effect sizes were generally smaller (we attribute this to the fact that single-task random cross-validation models achieved very high median AUC values that left little room for improvement).

There were some differences between our temporal validation and random cross-validation strategies that could lead to unfair comparisons; for instance,

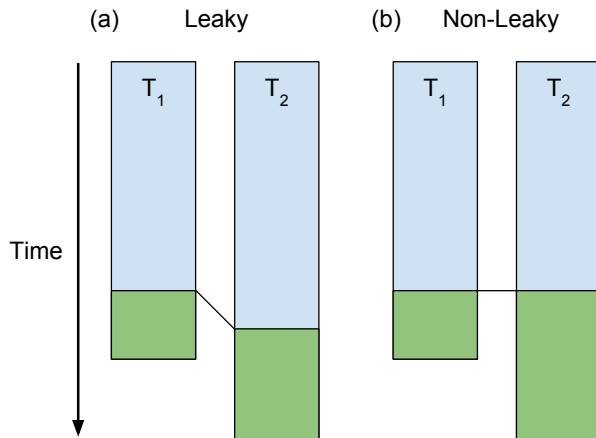


FIGURE 4: Leaky and non-leaky temporal validation of multitask models. In both strategies, datasets for two tasks (T_1 and T_2) are divided into training (blue) and test (green) data using specific time cutoffs (validation data is omitted for clarity). (a) Leaky validation divides each dataset independently, such that training data for one task could be in the future relative to training data for another task. (b) Non-leaky validation uses the same cutoff dates for all datasets.

random cross-validation models were trained on more data (80% of each dataset *vs.* 70% for temporal models) and we did not use held-out validation sets to prevent overfitting. To investigate the potential bias due to overfitting, we recalculated performance metrics for random cross-validation models using an alternative evaluation strategy that maximized the 5-fold mean AUC for each task rather than evaluating after a fixed number of training steps; these results are given in Section B.5. This alternative evaluation strategy revealed that random cross-validation models (especially STNNs) suffered from overfitting when evaluated at ~ 1 M steps, resulting in larger multitask effect sizes and more biased sign test confidence intervals when comparing MTNNs with STNNs. However, both strategies showed consistent multitask improvements relative to logistic regression, random forest, and STNN models.

These results indicate that multitask effects for specific models can vary depending on whether temporal validation or random cross-validation is used to estimate model performance, potentially leading to inconsistent conclusions regarding specific model architectures.



FIGURE 5: Distributions of maximum circular fingerprint Tanimoto similarity between each test compound and all training compounds (within each dataset) for temporal validation or random cross-validation. Note that the temporal validation histogram does not include similarities for validation set compounds.

4 Discussion

This report describes our efforts to apply multitask neural networks to industrial datasets for ADMET targets at Vertex Pharmaceuticals. Our results confirm previous work demonstrating that neural networks can outperform random forest and logistic regression baselines and provide further evidence that multitask learning can improve performance over single-task models.

Training models with large amounts of side information did not yield substantial improvements relative to simpler multitask models; comparisons of results on the 22 datasets in Table 1 did not show consistent improvements for models trained with 500+ tasks *vs.* 22 tasks, although some individual tasks did see benefits from additional side information. One possibility is that multitask learning on our datasets is quickly subject to diminishing returns, reminiscent of the “growth curves” presented by Ramsundar et al. [2015]. Furthermore, the role of task relatedness in multitask learning remains poorly understood, and it is possible that our datasets are simply too dissimilar to the additional side information to recapitulate the benefits seen in other studies.

Although our comparisons of temporal validation and random cross-validation were not ideal, our experiments show that random cross-validation can lead to overly optimistic estimates of multitask perfor-

mance (especially regarding the relative performance of specific model architectures). However, random cross-validation and temporal validation were generally in agreement about broader trends in performance (e.g. multitask models can outperform single-task models in both validation paradigms).

Practically speaking, we have not identified any universally superior method or architecture that will give optimal performance for every dataset. Accordingly, we suggest that that recommended best practices should be treated as starting points for building *dataset-specific* models, especially in production settings where it pays to have the best possible model for each individual task.

We expect that additional dataset curation and further exploration of model architectures will yield modest improvements, but we are also enthusiastic about recent work to improve the input representation for virtual screening models. Recent work in this area includes graph-based representations for deep learning systems such as “neural graph fingerprints” [Duvenaud et al., 2015] and “molecular graph convolutions” [Kearnes et al., 2016]. Additional work on learning from three-dimensional representations and handling conformational heterogeneity will help to capture more of the relevant components of the systems and processes we are attempting to model.

Acknowledgments

We thank Patrick Riley, Kevin McCloskey, Georgia McGaughey, W. Patrick Walters, Bharath Ramsundar, and the Vertex Modeling & Informatics team for helpful discussion. We also thank Gary Smith for computational support. S.K. was supported by an internship at Vertex Pharmaceuticals Inc. and by Google Inc. S.K. and V.P. also acknowledge support from from NIH 5U19AI109662-02.

Version information

Submitted to the Journal of Computer-Aided Molecular Design. Comments on arXiv versions:

v2: Corrected the summary in the Discussion of models trained with additional side information.

v3: Updates in response to reviewer comments: a) deemphasized random cross-validation results due to non-ideal comparisons with temporal validation models, b) added plots of validation AUC *vs.* training step for models trained with side information, c) up-

dated table headers and figure axis labels for clarity, d) additional minor changes to the main text and appendix.

References

- Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. TensorFlow: Large-scale machine learning on heterogeneous systems. *Software available from tensorflow.org*, 2015.
- Rich Caruana. Multitask learning. *Machine learning*, 28(1):41–75, 1997.
- George Dahl. Deep Learning How I Did It: Merck 1st place interview. *No Free Hunch*, November 1, 2012. URL <http://tinyurl.com/n2putqv>.
- George E Dahl, Navdeep Jaitly, and Ruslan Salakhutdinov. Multi-task neural networks for QSAR predictions. *arXiv preprint arXiv:1406.1231*, 2014.
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *The Journal of Machine Learning Research*, 12:2121–2159, 2011.
- David K Duvenaud, Dougal Maclaurin, Jorge Iparraguirre, Rafael Bombarell, Timothy Hirzel, Alán Aspuru-Guzik, and Ryan P Adams. Convolutional networks on graphs for learning molecular fingerprints. In *Advances in Neural Information Processing Systems*, pages 2215–2223, 2015.
- Dumitru Erhan, Pierre-Jean L’Heureux, Shi Yi Yue, and Yoshua Bengio. Collaborative filtering on a family of biological targets. *Journal of chemical information and modeling*, 46(2):626–635, 2006.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- Ajay N Jain and Anthony Nicholls. Recommendations for evaluation of computational methods. *Journal of computer-aided molecular design*, 22(3-4):133–139, 2008.
- Steven Kearnes, Kevin McCloskey, Marc Berndl, Vijay Pande, and Patrick Riley. Molecular graph convolutions: Moving beyond fingerprints. *arXiv preprint arXiv:1603.00856*, 2016.

- Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- Junshui Ma, Robert P Sheridan, Andy Liaw, George Dahl, and Vladimir Svetnik. Deep neural nets as a method for quantitative structure-activity relationships. *Journal of Chemical Information and Modeling*, 2015.
- Andreas Mayr, Günter Klambauer, Thomas Unterthiner, and Sepp Hochreiter. DeepTox: Toxicity prediction using deep learning. *Frontiers in Environmental Science*, 3:80, 2015.
- Georgia McGaughey, W Patrick Walters, and Brian Goldman. Understanding covariate shift in model performance [version 1; referees: 2 approved with reservations]. *F1000Research*, 5(Chem Inf Sci):597, 2016.
- OpenEye GraphSim Toolkit. URL <http://www.eyesopen.com>. OpenEye Scientific Software, Santa Fe, NM.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *The Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Bharath Ramsundar, Steven Kearnes, Patrick Riley, Dale Webster, David Konerding, and Vijay Pande. Massively multitask networks for drug discovery. *arXiv preprint arXiv:1502.02072*, 2015.
- David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of chemical information and modeling*, 50(5):742–754, 2010.
- Skipper Seabold and Josef Perktold. Statsmodels: Econometric and statistical modeling with python. In *Proceedings of the 9th Python in Science Conference*, pages 57–61, 2010.
- Robert P Sheridan. Time-split cross-validation as a method for estimating the goodness of prospective prediction. *Journal of chemical information and modeling*, 53(4):783–790, 2013.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- Thomas Unterthiner, Andreas Mayr, G Klambauer, Marvin Steijaert, Jörg K Wegner, Hugo Ceulemans, and Sepp Hochreiter. Deep learning as an opportunity in virtual screening. In *Proceedings of the Deep Learning Workshop at NIPS*, 2014.

APPENDIX

A Appendix: Methods

A.1 Datasets

We included side information from proprietary and public sources. An “internal” collection of over 3000 additional Vertex assays with μM readouts was downloaded from an internal database. These assays were further divided by readout type—such as IC_{50} , EC_{50} and K_i —yielding over 4000 datasets. Compounds with activities $\leq 1 \mu\text{M}$ were considered active, and activities $\geq 10 \mu\text{M}$ were considered inactive (intermediate activities were discarded).

We used public data from PubChem BioAssay (PCBA) [Wang et al., 2012] and the enhanced directory of useful decoys (DUD-E) [Mysinger et al., 2012], similar to the approach taken by Ramsundar et al. [2015]. We downloaded 128 PCBA datasets and used their associated active/inactive annotations without modification. All 102 DUD-E datasets were used, and each unique SMILES string was treated as a separate example (regardless of compound identifier) since many identifiers were associated with SMILES. For all datasets, multiple labels for the same compound were merged if they agreed and discarded otherwise (for the “internal” datasets, multiple μM values were averaged before assigning an activity class).

We employed a few simple filtering rules to limit the number of tasks in our multitask models. All tasks were required to have at least 10 actives and 10 inactives in their training set. Additionally, side information datasets (internal, PCBA, and DUD-E) were required to have at least 1000 training examples.

For temporal models utilizing public side information, PCBA datasets were tagged with their most recent modification date (Table A1) and all the DUD-E datasets were assigned to 14 July 2012 (the most recent modification date of the DUD-E tarball). Note, however, that since these models used *leaky* temporal validation (see Section 3.3), these datasets were used in their entirety as training data.

TABLE A1: Assay identification numbers (AIDs) and modification dates for PCBA datasets.

AID	Modification Date	AID	Modification Date	AID	Modification Date
411	2010-03-03	2528	2010-03-15	602179	2012-04-05
875	2010-07-12	2546	2010-03-13	602233	2012-01-27
881	2011-03-04	2549	2010-03-15	602310	2012-11-27
883	2010-07-06	2551	2010-03-15	602313	2012-03-08
884	2010-07-06	2662	2010-09-28	602332	2012-03-13
885	2010-07-06	2675	2014-02-22	624170	2012-05-22
887	2010-07-06	2676	2010-03-25	624171	2012-05-22
891	2010-07-06	463254	2010-09-23	624173	2012-05-30
899	2010-07-06	485281	2010-09-27	624202	2012-05-24
902	2010-07-06	485290	2010-09-28	624246	2012-06-01
903	2010-07-06	485294	2010-09-29	624287	2012-06-12
904	2010-07-06	485297	2010-09-29	624288	2013-12-14
912	2010-07-12	485313	2010-10-04	624291	2012-06-13
914	2010-07-06	485314	2010-09-29	624296	2012-06-15
915	2010-07-06	485341	2010-10-04	624297	2012-06-15
924	2010-07-06	485349	2010-10-04	624417	2012-07-26
925	2010-07-06	485353	2010-11-12	651635	2012-10-18
926	2010-07-06	485360	2010-10-05	651644	2012-10-16
927	2008-06-03	485364	2010-10-06	651768	2012-11-16
938	2010-07-06	485367	2010-11-05	651965	2013-01-03
995	2010-07-06	492947	2010-11-29	652025	2013-02-12
1030	2010-03-15	493208	2011-02-14	652104	2013-03-14
1379	2010-07-06	504327	2011-02-22	652105	2013-03-14
1452	2008-12-19	504332	2011-02-22	652106	2013-03-14
1454	2008-12-19	504333	2011-02-22	686970	2013-05-09
1457	2008-12-23	504339	2011-02-22	686978	2013-05-16
1458	2008-12-23	504444	2011-03-15	686979	2013-05-16
1460	2008-12-30	504466	2011-03-16	720504	2013-07-09
1461	2009-06-10	504467	2011-03-16	720532	2013-07-20
1468	2008-12-30	504706	2011-04-27	720542	2013-07-27
1469	2008-12-31	504842	2011-06-23	720551	2013-07-31
1471	2009-03-20	504845	2011-06-23	720553	2013-07-31
1479	2009-01-07	504847	2011-06-24	720579	2013-08-21
1631	2009-06-19	504891	2012-05-22	720580	2013-08-19
1634	2009-06-19	540276	2011-11-18	720707	2013-10-31
1688	2009-04-21	540317	2011-07-28	720708	2013-10-31
1721	2010-03-16	588342	2011-09-07	720709	2013-10-31
2100	2010-03-30	588453	2011-10-05	720711	2013-10-31
2101	2010-11-02	588456	2011-10-06	743255	2014-01-16
2147	2009-11-19	588579	2011-10-20	743266	2014-01-30
2242	2010-01-12	588590	2011-10-20		
2326	2010-06-16	588591	2011-10-20		
2451	2010-04-28	588795	2011-11-16		
2517	2013-07-16	588855	2011-12-06		

B Appendix: Results

B.1 Model performance

TABLE A2: Pairwise comparisons between neural network model architectures. For each pair of models within a model class (e.g. STNN), we report the median ΔAUC and sign test 95% confidence interval. Bold values indicate confidence intervals that do not include 0.5.

	Model A	Model B	Model B - Model A	
			Median ΔAUC	Sign Test 95% CI
STNN	(1000)	(4000)	0.006	(0.61, 0.93)
	(1000)	(2000, 100)	−0.004	(0.23, 0.61)
	(1000)	(2000, 1000)	−0.004	(0.20, 0.57)
	(1000)	(4000, 2000, 1000, 1000)	−0.004	(0.27, 0.65)
	(4000)	(2000, 100)	−0.006	(0.13, 0.48)
	(4000)	(2000, 1000)	−0.006	(0.20, 0.57)
	(4000)	(4000, 2000, 1000, 1000)	−0.011	(0.13, 0.48)
	(2000, 100)	(2000, 1000)	0.002	(0.35, 0.73)
	(2000, 100)	(4000, 2000, 1000, 1000)	−0.001	(0.28, 0.68)
	(2000, 1000)	(4000, 2000, 1000, 1000)	−0.006	(0.16, 0.53)
U-MTNN	(1000)	(4000)	−0.002	(0.31, 0.69)
	(1000)	(2000, 100)	−0.005	(0.27, 0.65)
	(1000)	(2000, 1000)	0.012	(0.52, 0.87)
	(1000)	(4000, 2000, 1000, 1000)	−0.004	(0.23, 0.61)
	(4000)	(2000, 100)	−0.009	(0.16, 0.53)
	(4000)	(2000, 1000)	0.006	(0.43, 0.80)
	(4000)	(4000, 2000, 1000, 1000)	−0.005	(0.27, 0.65)
	(2000, 100)	(2000, 1000)	0.012	(0.52, 0.87)
	(2000, 100)	(4000, 2000, 1000, 1000)	0.011	(0.52, 0.87)
	(2000, 1000)	(4000, 2000, 1000, 1000)	−0.009	(0.16, 0.53)
W-MTNN	(1000)	(4000)	−0.001	(0.27, 0.65)
	(1000)	(2000, 100)	−0.016	(0.13, 0.48)
	(1000)	(2000, 1000)	0.007	(0.52, 0.87)
	(1000)	(4000, 2000, 1000, 1000)	−0.007	(0.23, 0.61)
	(4000)	(2000, 100)	−0.009	(0.23, 0.61)
	(4000)	(2000, 1000)	0.005	(0.47, 0.84)
	(4000)	(4000, 2000, 1000, 1000)	−0.005	(0.23, 0.61)
	(2000, 100)	(2000, 1000)	0.020	(0.57, 0.90)
	(2000, 100)	(4000, 2000, 1000, 1000)	0.016	(0.57, 0.90)
	(2000, 1000)	(4000, 2000, 1000, 1000)	−0.008	(0.16, 0.53)

B.2 Factors affecting multitask learning

B.2.1 Individual dataset size



FIGURE A1: Box plots showing ΔAUC values between “subset” MTNN models trained only on small datasets ($<10\,000$ data points) and full MTNN models with the same core architecture. Each box plot summarizes 10 ΔAUC values, one for each combination of model architecture (e.g. (2000, 1000)) and task weighting strategy (U-MTNN or W-MTNN).



FIGURE A2: Differences between W-MTNN and U-MTNN test set AUC values for models with the same core architecture as a function of dataset size. Each point is the mean difference across all architectures; 95% confidence intervals were calculated by bootstrapping.

TABLE A3: Comparisons between “subset” models trained only on small datasets (<10 000 data points) and “full” multitask models trained on all 22 datasets in Table 1. We report the median ΔAUC and sign test 95% confidence interval comparing subset and full multitask models with the same core architecture and task weighting strategy. Differences were calculated only for the 16 datasets that were used to build subset models. Bold values indicate confidence intervals that do not include 0.5.

		Subset MTNN - Full MTNN	
	Model	Median ΔAUC	Sign Test 95% CI
U-MTNN	(1000)	0.005	(0.44, 0.86)
	(4000)	−0.003	(0.23, 0.67)
	(2000, 100)	−0.006	(0.18, 0.61)
	(2000, 1000)	−0.010	(0.14, 0.56)
	(4000, 2000, 1000, 1000)	−0.014	(0.10, 0.49)
W-MTNN	(1000)	−0.007	(0.14, 0.56)
	(4000)	−0.003	(0.18, 0.61)
	(2000, 100)	−0.009	(0.23, 0.67)
	(2000, 1000)	−0.009	(0.14, 0.56)
	(4000, 2000, 1000, 1000)	−0.007	(0.10, 0.49)

B.2.2 Total amount of data

TABLE A4: Median test set AUC values for MTNN models trained with additional side information (SI). We also report median ΔAUC values and sign test 95% confidence intervals for comparisons between each model and random forest or logistic regression. Bold values indicate confidence intervals that do not include 0.5.

			MTNN - Random Forest		MTNN - Logistic Regression	
	Model	Median AUC	Median ΔAUC	Sign Test 95% CI	Median ΔAUC	Sign Test 95% CI
U-MTNN +SI	(1000)	0.773	0.030	(0.47, 0.84)	0.003	(0.31, 0.69)
	(4000)	0.824	0.058	(0.61, 0.93)	0.035	(0.57, 0.90)
	(2000, 100)	0.772	0.002	(0.35, 0.73)	−0.019	(0.23, 0.61)
	(2000, 1000)	0.809	0.047	(0.61, 0.93)	0.021	(0.39, 0.77)
	(4000, 2000, 1000, 1000)	0.783	0.044	(0.47, 0.84)	0.016	(0.39, 0.77)
W-MTNN +SI	(1000)	0.816	0.061	(0.67, 0.95)	0.035	(0.61, 0.93)
	(4000)	0.808	0.059	(0.67, 0.95)	0.032	(0.67, 0.95)
	(2000, 100)	0.825	0.054	(0.67, 0.95)	0.035	(0.57, 0.90)
	(2000, 1000)	0.840	0.072	(0.78, 0.99)	0.059	(0.61, 0.93)
	(4000, 2000, 1000, 1000)	0.837	0.074	(0.67, 0.95)	0.062	(0.57, 0.90)



FIGURE A3: Validation set AUC for each task in multitask models trained with side information (U-MTNN+SI) as a function of training step.



FIGURE A4: Validation set AUC for each task in task-weighted multitask models trained with side information (W-MTNN+SI) as a function of training step.

TABLE A5: Comparisons between neural network models trained with additional side information (SI). Differences between STNN, U-MTNN+SI, and W-MTNN+SI models with the same core architecture are reported as median ΔAUC values and sign test 95% confidence intervals. Bold values indicate confidence intervals that do not include 0.5.

Model		MTNN+SI - STNN		W-MTNN+SI - U-MTNN+SI	
		Median ΔAUC	Sign Test 95% CI	Median ΔAUC	Sign Test 95% CI
U-MTNN +SI	(1000)	-0.008	(0.27, 0.65)		
	(4000)	0.008	(0.43, 0.80)		
	(2000, 100)	-0.024	(0.27, 0.65)		
	(2000, 1000)	0.018	(0.39, 0.77)		
	(4000, 2000, 1000, 1000)	0.015	(0.39, 0.77)		
W-MTNN +SI	(1000)	0.025	(0.39, 0.77)	0.034	(0.78, 0.99)
	(4000)	0.007	(0.52, 0.87)	0.005	(0.47, 0.84)
	(2000, 100)	0.033	(0.43, 0.80)	0.048	(0.72, 0.97)
	(2000, 1000)	0.045	(0.61, 0.93)	0.020	(0.67, 0.95)
	(4000, 2000, 1000, 1000)	0.043	(0.57, 0.90)	0.036	(0.52, 0.87)



FIGURE A5: Differences in test set AUC values between MTNN models trained with and without side information (SI). Each box plot summarizes 10 ΔAUC values, one for each combination of model architecture (e.g. (2000, 1000)) and task weighting strategy (U-MTNN or W-MTNN).

B.2.3 Task relatedness

TABLE A6: Datasets used to construct subset multitask models. Each subset multitask model used a subset of the datasets from Table 1. The datasets in each subset are related by a similar assay target.

Subset	Datasets
Solubility	B, C
Metabolism	D, E, F
Stability	N, O, Q
Clearance	T, U

TABLE A7: Comparisons between subset and full multitask models. We report the median Δ AUC and sign test 95% confidence interval comparing subset and full multitask models with the same core architecture and task weighting strategy. Differences were calculated only for the 10 datasets that were used to build subset models (see Table A6). Bold values indicate confidence intervals that do not include 0.5.

		Subset MTNN - Full MTNN	
Model		Median Δ AUC	Sign Test 95% CI
U-MTNN	(1000)	-0.005	(0.11, 0.60)
	(4000)	-0.005	(0.06, 0.51)
	(2000, 100)	-0.009	(0.17, 0.69)
	(2000, 1000)	-0.015	(0.17, 0.69)
	(4000, 2000, 1000, 1000)	-0.016	(0.17, 0.69)
W-MTNN	(1000)	-0.005	(0.06, 0.51)
	(4000)	-0.006	(0.11, 0.60)
	(2000, 100)	-0.008	(0.17, 0.69)
	(2000, 1000)	-0.015	(0.11, 0.60)
	(4000, 2000, 1000, 1000)	-0.020	(0.17, 0.69)

Multitask learning can only improve performance when tasks are related [Caruana, 1997]. Unfortunately, the concept of *relatedness* is not well defined for chemical datasets. Any measure of task relatedness must capture the degree to which two datasets follow the *similar property principle* [Willett, 2009] for ligand-based virtual screening: compounds with similar properties should have similar behavior. Hence, both the features used as input to the model (molecular descriptors) and the training labels (experimental behavior) must be considered. Previous attempts to quantify relatedness have focused on shared compounds or correlations between labels [Erhan et al., 2006; Ramsundar et al., 2015]. We attempted to construct a new metric for task relatedness based on correlation between labels for similar compounds in different tasks (described below), but did not fully explore any relationship between this measure of task relatedness and multitask improvement in temporal validation.

We use Tanimoto similarity between compound fingerprints to calculate a measure of dataset relatedness R under the assumption that similar compounds will have similar labels in related tasks. Given two datasets, α and β , we calculate the Tanimoto similarity ($T_{a,b}$) for each pair of compounds ($a \in \alpha, b \in \beta$). For pairs above a similarity threshold τ , we count the number of pairs with the same (S) or different (D) labels. Since label meanings are somewhat arbitrary—related tasks may have correlated or anticorrelated labels—we then take the maximum of S and D and normalize it by the number of similar pairs (non-similar pairs provide



FIGURE A6: Box plots showing ΔAUC values between subset and full models with the same core architecture. Each box plot summarizes 10 ΔAUC values, one for each combination of model architecture (e.g. (2000, 1000)) and task weighting strategy (U-MTNN or W-MTNN).

no information about task relatedness). Formally:

$$S(\alpha, \beta) = \sum_{a \in \alpha, b \in \beta} \mathbb{1}(T_{a,b} \geq \tau) \mathbb{1}(y_a = y_b) \quad (1)$$

$$D(\alpha, \beta) = \sum_{a \in \alpha, b \in \beta} \mathbb{1}(T_{a,b} \geq \tau) \mathbb{1}(y_a \neq y_b) \quad (2)$$

$$R(\alpha, \beta) = \frac{\max\{S(\alpha, \beta), D(\alpha, \beta)\}}{S(\alpha, \beta) + D(\alpha, \beta)} \quad (3)$$

Note that this metric is symmetric and takes into account both the number of similar compounds and the correlation of their labels. It ranges from 0.5 to 1.0, with unity indicating identical datasets. We used $\tau = 0.5$, in accordance with the similarity/dissimilarity threshold for ECFP4 estimated by [Franco et al. \[2014\]](#). This measure of relatedness has some shortcomings, including a tendency to give nonintuitive results when datasets have very few similar pairs.

One potential problem with attempts to correlate any measure of task relatedness with multitask improvement is that, in general, multitask improvements are relatively small ($< 10\%$) and dataset-dependent, which makes it somewhat dangerous to look for correlations at all.

B.3 Information leakage in multitask networks

TABLE A8: Median test set AUC values for models using non-leaky temporal validation. We also report median Δ AUC values and sign test 95% confidence intervals for comparisons between each model and random forest or logistic regression. Bold values indicate confidence intervals that do not include 0.5.

Model	Median AUC	Model - Random Forest		Model - Logistic Regression	
		Median Δ AUC	Sign Test 95% CI	Median Δ AUC	Sign Test 95% CI
U-MTNN	(1000)	0.793	0.056 (0.67, 0.95)	0.036	(0.57, 0.90)
	(4000)	0.789	0.055 (0.67, 0.95)	0.029	(0.57, 0.90)
	(2000, 100)	0.759	0.040 (0.57, 0.90)	0.014	(0.47, 0.84)
	(2000, 1000)	0.778	0.049 (0.65, 0.95)	0.037	(0.61, 0.93)
	(4000, 2000, 1000, 1000)	0.784	0.055 (0.72, 0.97)	0.033	(0.47, 0.84)
W-MTNN	(1000)	0.793	0.059 (0.61, 0.93)	0.039	(0.67, 0.95)
	(4000)	0.783	0.058 (0.61, 0.93)	0.025	(0.67, 0.95)
	(2000, 100)	0.761	0.034 (0.61, 0.93)	0.024	(0.43, 0.80)
	(2000, 1000)	0.782	0.055 (0.67, 0.95)	0.040	(0.61, 0.93)
	(4000, 2000, 1000, 1000)	0.785	0.051 (0.61, 0.93)	0.039	(0.47, 0.84)

TABLE A9: Comparisons between neural network models using non-leaky temporal validation. Differences between STNN, U-MTNN, and W-MTNN models with the same core architecture are reported as median Δ AUC values and sign test 95% confidence intervals. Bold values indicate confidence intervals that do not include 0.5.

Model	Median Δ AUC	MTNN - STNN		W-MTNN - U-MTNN	
		Median Δ AUC	Sign Test 95% CI	Median Δ AUC	Sign Test 95% CI
U-MTNN	(1000)	0.009	(0.57, 0.90)		
	(4000)	0.004	(0.43, 0.80)		
	(2000, 100)	0.012	(0.47, 0.84)		
	(2000, 1000)	0.027	(0.57, 0.90)		
	(4000, 2000, 1000, 1000)	0.029	(0.43, 0.80)		
W-MTNN	(1000)	0.017	(0.67, 0.95)	0.000	(0.31, 0.69)
	(4000)	0.008	(0.52, 0.87)	0.001	(0.35, 0.73)
	(2000, 100)	0.015	(0.39, 0.77)	0.005	(0.43, 0.80)
	(2000, 1000)	0.026	(0.52, 0.87)	-0.001	(0.27, 0.65)
	(4000, 2000, 1000, 1000)	0.032	(0.47, 0.84)	0.011	(0.52, 0.87)

TABLE A10: Pairwise comparisons between neural network model architectures using non-leaky temporal validation. For each pair of models within a model class (e.g. STNN), we report the median ΔAUC and sign test 95% confidence interval. Bold values indicate confidence intervals that do not include 0.5.

	Model A	Model B	Model B - Model A	
			Median ΔAUC	Sign Test 95% CI
STNN	(1000)	(4000)	0.006	(0.61, 0.93)
	(1000)	(2000, 100)	-0.004	(0.23, 0.61)
	(1000)	(2000, 1000)	-0.004	(0.20, 0.57)
	(1000)	(4000, 2000, 1000, 1000)	-0.004	(0.27, 0.65)
	(4000)	(2000, 100)	-0.006	(0.13, 0.48)
	(4000)	(2000, 1000)	-0.006	(0.20, 0.57)
	(4000)	(4000, 2000, 1000, 1000)	-0.011	(0.13, 0.48)
	(2000, 100)	(2000, 1000)	0.002	(0.35, 0.73)
	(2000, 100)	(4000, 2000, 1000, 1000)	-0.001	(0.28, 0.68)
	(2000, 1000)	(4000, 2000, 1000, 1000)	-0.006	(0.16, 0.53)
U-MTNN	(1000)	(4000)	-0.001	(0.27, 0.65)
	(1000)	(2000, 100)	-0.012	(0.23, 0.61)
	(1000)	(2000, 1000)	0.004	(0.43, 0.80)
	(1000)	(4000, 2000, 1000, 1000)	-0.001	(0.27, 0.65)
	(4000)	(2000, 100)	-0.008	(0.23, 0.61)
	(4000)	(2000, 1000)	0.008	(0.43, 0.80)
	(4000)	(4000, 2000, 1000, 1000)	-0.002	(0.27, 0.65)
	(2000, 100)	(2000, 1000)	0.006	(0.43, 0.80)
	(2000, 100)	(4000, 2000, 1000, 1000)	0.007	(0.57, 0.90)
	(2000, 1000)	(4000, 2000, 1000, 1000)	-0.001	(0.27, 0.65)
W-MTNN	(1000)	(4000)	-0.003	(0.23, 0.61)
	(1000)	(2000, 100)	-0.006	(0.16, 0.53)
	(1000)	(2000, 1000)	0.003	(0.39, 0.77)
	(1000)	(4000, 2000, 1000, 1000)	-0.005	(0.20, 0.57)
	(4000)	(2000, 100)	0.000	(0.31, 0.69)
	(4000)	(2000, 1000)	0.009	(0.39, 0.77)
	(4000)	(4000, 2000, 1000, 1000)	0.002	(0.31, 0.69)
	(2000, 100)	(2000, 1000)	0.015	(0.52, 0.87)
	(2000, 100)	(4000, 2000, 1000, 1000)	0.016	(0.47, 0.84)
	(2000, 1000)	(4000, 2000, 1000, 1000)	-0.007	(0.23, 0.61)



FIGURE A7: Box plots showing ΔAUC values between non-leaky MTNN models and STNN models with the same core architecture. Each box plot summarizes 10 ΔAUC values, one for each combination of model architecture (e.g. (2000, 1000)) and task weighting strategy (U-MTNN or W-MTNN).



FIGURE A8: Box plots showing ΔAUC values between non-leaky and leaky models with the same core architecture. Each box plot summarizes 10 ΔAUC values, one for each combination of model architecture (e.g. (2000, 1000)) and task weighting strategy (U-MTNN or W-MTNN).

B.4 Random cross-validation

TABLE A11: Median 5-fold mean test set AUC values for models using random cross-validation. We also report median Δ AUC values and sign test 95% confidence intervals for comparisons between each model and random forest or logistic regression. Bold values indicate confidence intervals that do not include 0.5.

Model	Median AUC	Model - Random Forest		Model - Logistic Regression	
		Median Δ AUC	Sign Test 95% CI	Median Δ AUC	Sign Test 95% CI
Random Forest	0.916			0.022	(0.67, 0.95)
Logistic Regression	0.896	-0.022	(0.05, 0.33)		
STNN	(1000)	0.923	0.005 (0.43, 0.80)	0.021	(0.57, 0.90)
	(4000)	0.926	(0.61, 0.93)	0.025	(0.67, 0.95)
	(2000, 100)	0.918	0.005 (0.47, 0.84)	0.024	(0.61, 0.93)
	(2000, 1000)	0.915	0.002 (0.35, 0.73)	0.018	(0.57, 0.90)
	(4000, 2000, 1000, 1000)	0.908	-0.005 (0.27, 0.65)	0.009	(0.35, 0.73)
U-MTNN	(1000)	0.930	(0.85, 1.00)	0.040	(0.85, 1.00)
	(4000)	0.931	(0.85, 1.00)	0.043	(0.85, 1.00)
	(2000, 100)	0.930	(0.72, 0.97)	0.034	(0.85, 1.00)
	(2000, 1000)	0.939	(0.85, 1.00)	0.051	(0.85, 1.00)
	(4000, 2000, 1000, 1000)	0.937	(0.85, 1.00)	0.042	(0.85, 1.00)
W-MTNN	(1000)	0.934	(0.85, 1.00)	0.041	(0.85, 1.00)
	(4000)	0.933	(0.85, 1.00)	0.043	(0.85, 1.00)
	(2000, 100)	0.931	(0.67, 0.95)	0.042	(0.78, 0.99)
	(2000, 1000)	0.936	(0.85, 1.00)	0.050	(0.78, 0.99)
	(4000, 2000, 1000, 1000)	0.937	(0.85, 1.00)	0.046	(0.78, 0.99)

TABLE A12: Comparisons between neural network models using random cross-validation. Differences between STNN, U-MTNN, and W-MTNN models with the same core architecture are reported as median ΔAUC values and sign test 95% confidence intervals. Bold values indicate confidence intervals that do not include 0.5.

Model		MTNN - STNN		W-MTNN - U-MTNN	
		Median ΔAUC	Sign Test 95% CI	Median ΔAUC	Sign Test 95% CI
U-MTNN	(1000)	0.012	(0.85, 1.00)		
	(4000)	0.011	(0.78, 0.99)		
	(2000, 100)	0.016	(0.57, 0.90)		
	(2000, 1000)	0.024	(0.85, 1.00)		
	(4000, 2000, 1000, 1000)	0.033	(0.85, 1.00)		
W-MTNN	(1000)	0.012	(0.67, 0.95)	0.001	(0.39, 0.77)
	(4000)	0.010	(0.85, 1.00)	0.000	(0.39, 0.77)
	(2000, 100)	0.015	(0.57, 0.90)	0.003	(0.35, 0.73)
	(2000, 1000)	0.025	(0.85, 1.00)	−0.001	(0.20, 0.57)
	(4000, 2000, 1000, 1000)	0.033	(0.72, 0.97)	0.000	(0.39, 0.77)



FIGURE A9: Box plots showing ΔAUC values between MTNN and STNN models with the same core architecture for models trained using random cross-validation. Each box plot summarizes 10 ΔAUC values, one for each combination of model architecture (e.g. (2000, 1000)) and task weighting strategy (U-MTNN or W-MTNN).

TABLE A13: Pairwise comparisons between neural network model architectures using random cross-validation. For each pair of models within a model class (e.g. STNN), we report the median ΔAUC and sign test 95% confidence interval. Bold values indicate confidence intervals that do not include 0.5.

	Model A	Model B	Model B - Model A	
			Median ΔAUC	Sign Test 95% CI
STNN	(1000)	(4000)	0.003	(0.67, 0.95)
	(1000)	(2000, 100)	0.001	(0.39, 0.77)
	(1000)	(2000, 1000)	-0.001	(0.13, 0.48)
	(1000)	(4000, 2000, 1000, 1000)	-0.007	(0.10, 0.43)
	(4000)	(2000, 100)	-0.002	(0.13, 0.48)
	(4000)	(2000, 1000)	-0.005	(0.01, 0.22)
	(4000)	(4000, 2000, 1000, 1000)	-0.010	(0.03, 0.28)
	(2000, 100)	(2000, 1000)	-0.003	(0.00, 0.15)
	(2000, 100)	(4000, 2000, 1000, 1000)	-0.008	(0.03, 0.28)
U-MTNN	(2000, 1000)	(4000, 2000, 1000, 1000)	-0.003	(0.16, 0.53)
	(1000)	(4000)	0.001	(0.43, 0.80)
	(1000)	(2000, 100)	-0.003	(0.20, 0.57)
	(1000)	(2000, 1000)	0.009	(0.78, 0.99)
	(1000)	(4000, 2000, 1000, 1000)	0.005	(0.47, 0.84)
	(4000)	(2000, 100)	-0.001	(0.27, 0.65)
	(4000)	(2000, 1000)	0.008	(0.85, 1.00)
	(4000)	(4000, 2000, 1000, 1000)	0.003	(0.57, 0.90)
	(2000, 100)	(2000, 1000)	0.010	(0.85, 1.00)
W-MTNN	(2000, 100)	(4000, 2000, 1000, 1000)	0.005	(0.67, 0.95)
	(2000, 1000)	(4000, 2000, 1000, 1000)	-0.003	(0.10, 0.43)
	(1000)	(4000)	0.000	(0.35, 0.73)
	(1000)	(2000, 100)	0.001	(0.35, 0.73)
	(1000)	(2000, 1000)	0.009	(0.67, 0.95)
	(1000)	(4000, 2000, 1000, 1000)	0.006	(0.52, 0.87)
	(4000)	(2000, 100)	-0.002	(0.23, 0.61)
	(4000)	(2000, 1000)	0.007	(0.57, 0.90)
	(4000)	(4000, 2000, 1000, 1000)	0.003	(0.35, 0.73)
	(2000, 100)	(2000, 1000)	0.006	(0.57, 0.90)
	(2000, 100)	(4000, 2000, 1000, 1000)	0.004	(0.67, 0.95)
	(2000, 1000)	(4000, 2000, 1000, 1000)	-0.002	(0.20, 0.57)

B.5 Random cross-validation: Alternative evaluation strategy

Because we did not hold out a validation set for checkpoint selection when training random cross-validation models, the values reported in Section B.4 were calculated from the final training checkpoint of each per-fold model. In this section, we report values generated using the per-fold checkpoints closest to a “target step” that maximized the 5-fold mean AUC for each task. Note that target steps were chosen for each task independently.

TABLE A14: Median 5-fold mean test set AUC values for models using random cross-validation with target step evaluation. We also report median Δ AUC values and sign test 95% confidence intervals for comparisons between each model and random forest or logistic regression. Bold values indicate confidence intervals that do not include 0.5.

Model	Median AUC	Model - Random Forest		Model - Logistic Regression	
		Median Δ AUC	Sign Test 95% CI	Median Δ AUC	Sign Test 95% CI
Random Forest	0.916			0.022	(0.67, 0.95)
Logistic Regression	0.896	-0.022	(0.05, 0.33)		
STNN	(1000)	0.928	0.009 (0.72, 0.97)	0.026	(0.72, 0.97)
	(4000)	0.931	0.010 (0.72, 0.97)	0.028	(0.78, 0.99)
	(2000, 100)	0.923	0.011 (0.61, 0.93)	0.028	(0.72, 0.97)
	(2000, 1000)	0.929	0.009 (0.67, 0.95)	0.028	(0.72, 0.97)
	(4000, 2000, 1000, 1000)	0.924	0.010 (0.67, 0.95)	0.028	(0.67, 0.95)
U-MTNN	(1000)	0.931	0.021 (0.85, 1.00)	0.040	(0.85, 1.00)
	(4000)	0.932	0.021 (0.85, 1.00)	0.043	(0.85, 1.00)
	(2000, 100)	0.931	0.019 (0.72, 0.97)	0.035	(0.85, 1.00)
	(2000, 1000)	0.940	0.028 (0.85, 1.00)	0.052	(0.85, 1.00)
	(4000, 2000, 1000, 1000)	0.939	0.025 (0.85, 1.00)	0.045	(0.85, 1.00)
W-MTNN	(1000)	0.935	0.019 (0.85, 1.00)	0.041	(0.85, 1.00)
	(4000)	0.934	0.020 (0.85, 1.00)	0.044	(0.85, 1.00)
	(2000, 100)	0.933	0.023 (0.78, 0.99)	0.042	(0.85, 1.00)
	(2000, 1000)	0.941	0.029 (0.85, 1.00)	0.051	(0.85, 1.00)
	(4000, 2000, 1000, 1000)	0.939	0.025 (0.85, 1.00)	0.048	(0.85, 1.00)

TABLE A15: Comparisons between neural network models using random cross-validation with target step evaluation. Differences between STNN, U-MTNN, and W-MTNN models with the same core architecture are reported as median ΔAUC values and sign test 95% confidence intervals. Bold values indicate confidence intervals that do not include 0.5.

Model		MTNN - STNN		W-MTNN - U-MTNN	
		Median ΔAUC	Sign Test 95% CI	Median ΔAUC	Sign Test 95% CI
U-MTNN	(1000)	0.010	(0.61, 0.93)		
	(4000)	0.009	(0.72, 0.97)		
	(2000, 100)	0.013	(0.52, 0.87)		
	(2000, 1000)	0.017	(0.85, 1.00)		
	(4000, 2000, 1000, 1000)	0.015	(0.78, 0.99)		
W-MTNN	(1000)	0.010	(0.57, 0.90)	0.001	(0.47, 0.84)
	(4000)	0.008	(0.72, 0.97)	0.000	(0.39, 0.77)
	(2000, 100)	0.013	(0.52, 0.87)	0.006	(0.47, 0.84)
	(2000, 1000)	0.016	(0.85, 1.00)	0.000	(0.27, 0.65)
	(4000, 2000, 1000, 1000)	0.015	(0.67, 0.95)	0.001	(0.35, 0.73)



FIGURE A10: Box plots showing ΔAUC values between MTNN and STNN models with the same core architecture for models trained using random cross-validation with target step evaluation. Each box plot summarizes 10 ΔAUC values, one for each combination of model architecture (e.g. (2000, 1000)) and task weighting strategy (U-MTNN or W-MTNN).

TABLE A16: Pairwise comparisons between neural network model architectures using random cross-validation with target step evaluation. For each pair of models within a model class (e.g. STNN), we report the median ΔAUC and sign test 95% confidence interval. Bold values indicate confidence intervals that do not include 0.5.

	Model A	Model B	Model B - Model A	
			Median ΔAUC	Sign Test 95% CI
STNN	(1000)	(4000)	0.002	(0.67, 0.95)
	(1000)	(2000, 100)	0.000	(0.39, 0.77)
	(1000)	(2000, 1000)	0.000	(0.27, 0.65)
	(1000)	(4000, 2000, 1000, 1000)	0.001	(0.35, 0.73)
	(4000)	(2000, 100)	−0.001	(0.10, 0.43)
	(4000)	(2000, 1000)	−0.001	(0.10, 0.43)
	(4000)	(4000, 2000, 1000, 1000)	−0.001	(0.10, 0.43)
	(2000, 100)	(2000, 1000)	0.000	(0.39, 0.77)
	(2000, 100)	(4000, 2000, 1000, 1000)	0.000	(0.35, 0.73)
	(2000, 1000)	(4000, 2000, 1000, 1000)	0.000	(0.23, 0.61)
U-MTNN	(1000)	(4000)	0.001	(0.47, 0.84)
	(1000)	(2000, 100)	−0.003	(0.20, 0.57)
	(1000)	(2000, 1000)	0.009	(0.78, 0.99)
	(1000)	(4000, 2000, 1000, 1000)	0.005	(0.57, 0.90)
	(4000)	(2000, 100)	−0.002	(0.27, 0.65)
	(4000)	(2000, 1000)	0.008	(0.85, 1.00)
	(4000)	(4000, 2000, 1000, 1000)	0.005	(0.57, 0.90)
	(2000, 100)	(2000, 1000)	0.010	(0.78, 0.99)
	(2000, 100)	(4000, 2000, 1000, 1000)	0.005	(0.72, 0.97)
	(2000, 1000)	(4000, 2000, 1000, 1000)	−0.002	(0.20, 0.57)
W-MTNN	(1000)	(4000)	0.000	(0.27, 0.65)
	(1000)	(2000, 100)	0.001	(0.35, 0.73)
	(1000)	(2000, 1000)	0.009	(0.72, 0.97)
	(1000)	(4000, 2000, 1000, 1000)	0.007	(0.67, 0.95)
	(4000)	(2000, 100)	0.000	(0.35, 0.73)
	(4000)	(2000, 1000)	0.007	(0.72, 0.97)
	(4000)	(4000, 2000, 1000, 1000)	0.004	(0.57, 0.90)
	(2000, 100)	(2000, 1000)	0.006	(0.67, 0.95)
	(2000, 100)	(4000, 2000, 1000, 1000)	0.004	(0.85, 1.00)
	(2000, 1000)	(4000, 2000, 1000, 1000)	−0.002	(0.23, 0.61)

References

- Rich Caruana. Multitask learning. *Machine learning*, 28(1):41–75, 1997.
- Dumitru Erhan, Pierre-Jean L’Heureux, Shi Yi Yue, and Yoshua Bengio. Collaborative filtering on a family of biological targets. *Journal of chemical information and modeling*, 46(2):626–635, 2006.
- Pedro Franco, Nuria Porta, John D Holliday, and Peter Willett. The use of 2D fingerprint methods to support the assessment of structural similarity in orphan drug legislation. *J. Cheminformatics*, 6(1):5, 2014.
- Michael M Mysinger, Michael Carchia, John J Irwin, and Brian K Shoichet. Directory of useful decoys, enhanced (DUD-E): better ligands and decoys for better benchmarking. *Journal of medicinal chemistry*, 55(14):6582–6594, 2012.
- Bharath Ramsundar, Steven Kearnes, Patrick Riley, Dale Webster, David Konerding, and Vijay Pande. Massively multitask networks for drug discovery. *arXiv preprint arXiv:1502.02072*, 2015.
- Yanli Wang, Jewen Xiao, Tugba O Suzek, Jian Zhang, Jiyao Wang, Zhigang Zhou, Lianyi Han, Karen Karapetyan, Svetlana Dracheva, Benjamin A Shoemaker, et al. PubChem’s BioAssay database. *Nucleic acids research*, 40(D1):D400–D412, 2012.
- Peter Willett. Similarity methods in chemoinformatics. *Annual review of information science and technology*, 43(1):1–117, 2009.